

Aula 08 - Coleção de objetos Conta

Contexto de negócio

Carteira de clientes: operar lista dinâmica de contas mantendo previsibilidade de leitura e escrita.

Uma conta única em objeto funciona, mas o sistema real precisa de várias contas.

Tensão operacional

Sem coleção, teríamos variáveis separadas para cada conta.

Objetivo técnico da entrega

- `ArrayList<Conta>`.
- Iteração `for-each`.
- Explicar `<>` (generics) do zero no contexto da aula.
- Comparar `array` vs `ArrayList` com impacto prático no fluxo.

Recurso técnico desta aula

Recurso inserido: lista de contas com inserção e operação por item.

Fundamento novo: o que são coleções

Coleção é uma estrutura para guardar vários elementos sob uma única variável.

Nesta aula, a coleção escolhida é `ArrayList`, que representa uma lista dinâmica.

Diferença conceitual importante:

- `array`: estrutura fixa em tamanho;
- `ArrayList`: estrutura que cresce e encolhe conforme o uso.

Pergunta para orientar o raciocínio:

- quando o volume de contas muda ao longo do dia, faz sentido depender de estrutura fixa?

Fundamento novo: o que significa `ArrayList<Conta>`

Essa linha traz duas ideias ao mesmo tempo:

```
ArrayList<Conta> contas = new ArrayList<>();
```

1. `ArrayList`

- é o tipo da coleção (lista dinâmica).

2. <Conta>

- é o tipo do elemento guardado na lista;
- em Java, isso se chama generic (generics);
- leitura prática: "esta lista só aceita Conta".

Impacto direto do <>:

- melhora segurança de tipo;
- reduz erro de uso com elementos incompatíveis;
- deixa o código mais legível para manutenção.

Problema que justifica o novo artefato

Com objetos `Conta` isolados, ainda há um gargalo:

- cada nova conta exige nova variável e novo trecho de operação;
- o algoritmo não escala bem para cadastro dinâmico;
- inclusão e remoção de contas ficam rígidas.

Por isso entra o `ArrayList<Conta>`: coleção dinâmica para crescer, percorrer e operar sem redesenhar a estrutura a cada nova conta.

Antes e depois da mudança (array vs ArrayList)

Antes (array):

```
Conta[] contas = new Conta[2];
contas[0] = new Conta(1, "Ana", 1000.00);
contas[1] = new Conta(2, "Bruno", 700.00);
```

Se precisar de terceira conta, surge um problema estrutural: tamanho fixo.

Depois (ArrayList):

```
ArrayList<Conta> contas = new ArrayList<>();
contas.add(new Conta(1, "Ana", 1000.00));
contas.add(new Conta(2, "Bruno", 700.00));
contas.add(new Conta(3, "Carla", 500.00));
```

O que muda de verdade:

- inclusão deixa de depender de capacidade fixa inicial;
- o fluxo principal passa a lidar melhor com cadastro dinâmico;
- o algoritmo fica mais próximo da operação real do sistema.

Artefato explicado: ArrayList no fluxo da aula

Papel de cada operação no exemplo:

- `new ArrayList<>()`: cria coleção vazia;
- `add(...)`: inclui nova conta no cadastro;
- `get(i)`: acessa conta por posição;
- `for (Conta conta : contas)`: percorre toda a coleção para execução em lote.

Leitura detalhada do fluxo:

- o cadastro nasce vazio;
- cada `add(...)` representa uma entrada de conta no sistema;
- operações (`depositar`, `sacar`) são chamadas em elementos da coleção;
- o `for-each` consolida visão operacional sem expor índice explicitamente no laço.

Riscos previstos para discussão:

- `get(i)` depende de posição e pode ficar frágil quando a ordem muda;
- acesso por índice sem validação pode gerar erro;
- por isso a próxima evolução busca conta por identificador, não por posição fixa.

Limites importantes para não romantizar o `ArrayList`:

- ele resolve crescimento dinâmico, mas não resolve sozinho identificação de conta;
- sem critério de busca, posição ainda pode induzir erro de negócio;
- coleção melhorou estrutura, porém ainda exige disciplina de acesso.

Demonstração prática do impacto

Cenário A (array fixo):

- abriu com capacidade 2;
- chegou terceira conta;
- fluxo exige estratégia extra para redimensionar/copiar.

Cenário B (ArrayList):

- abriu vazio;
- contas são adicionadas conforme entram;
- fluxo acompanha operação real com menor fricção estrutural.

Conclusão prática:

- o ganho principal não é sintaxe, é elasticidade operacional da estrutura de dados.

Referência direta do artefato:

- W3Schools ArrayList: https://www.w3schools.com/java/java_arraylist.asp
- W3Schools For Loop: https://www.w3schools.com/java/java_for_loop.asp

Transição didática para a próxima aula

Com coleção dinâmica estabelecida, o próximo problema é localizar conta com segurança sem depender de posição fixa. A próxima aula introduz busca por identificador.

Valor prático entregue

- Facilita crescimento do cadastro sem redefinir estrutura.
- A base fica pronta para evolução controlada da próxima capacidade.

Fluxo operacional explicado

- Preparar os dados de entrada do cenário da aula.
- Executar a regra principal e observar o impacto no estado.
- Operar a coleção de contas com validação de alvo e consistência de dados.
- Confirmar saída de sucesso, erro e borda antes de concluir a etapa.

Código em foco

```
import java.util.ArrayList;

class Conta {
    int id;
    String titular;
    double saldo;

    Conta(int p_id, String p_titular, double p_saldoInicial) {
        id = p_id;
        titular = p_titular;
        saldo = p_saldoInicial;
    }

    void depositar(double p_valor) {
        if (p_valor > 0) {
            saldo = saldo + p_valor;
        }
    }

    boolean sacar(double p_valor) {
        if (p_valor > 0 && p_valor <= saldo) {
            saldo = saldo - p_valor;
            return true;
        }
        return false;
    }

    @Override
    public String toString() {
        return "ID: " + id + " | Titular: " + titular + " | Saldo: " + saldo;
    }
}

// Classe temporaria de fluxo do programa.
```

```
// Neste curso, ela existe para iniciar a execução em Java.
public class AppGestaoContas {
    public static void main(String[] args) {
        // Aula 08: coleção de objetos.
        ArrayList<Conta> contas = new ArrayList<>();
        contas.add(new Conta(1, "Ana", 1000.00));
        contas.add(new Conta(2, "Bruno", 700.00));

        contas.get(0).depositar(100.00);
        contas.get(1).sacar(20.00);

        for (Conta conta : contas) {
            System.out.println(conta);
        }
    }
}
```

Leitura técnica orientada

Percorra o código com estes checkpoints de revisão:

- Estrutura de domínio: identificar onde a entidade de negócio concentra dados.
- Regra de decisão: mapear condições que aprovam ou negam operações.
- Estrutura de dados: justificar uso de lista dinâmica no contexto da aula.
- Saída observável: conferir mensagens que comprovam sucesso, erro e bloqueio de regra.

Discussão de contrato de retorno

Quando o método retorna **boolean**, o contrato precisa ser tratado como regra de negócio:

- **true** significa operação aprovada e estado atualizado;
- **false** significa operação negada e estado preservado.

Risco de lógica silenciosa:

- ignorar o **false** no chamador cria fluxo aparentemente correto, mas operacionalmente incompleto.

Responsabilidade de quem chama:

- sempre tratar os dois ramos (**true** e **false**) com mensagens e ações coerentes;
- registrar motivo de negativa quando possível (conta inativa, valor inválido, limite excedido);
- evitar seguir o fluxo como se a operação tivesse ocorrido.

Variações para debate técnico:

1. Em quais cenários **boolean** basta?
2. Quando o time precisa de retorno com motivo da falha?
3. Qual custo de não tratar o **false** em produção?

Comparativo de paradigma: procedural vs POO

Diferença que precisa ficar nítida neste ponto da trilha:

- Procedural: o estado é passado entre funções por parâmetros.
- POO: o estado pertence ao objeto, e os métodos operam nesse estado interno.

Impacto no desenho do algoritmo:

- no procedural, o algoritmo coordena dados externos e funções;
- na POO, o algoritmo delega comportamento para objetos com responsabilidades claras.

Critério de maturidade técnica:

- reconhecer quando o custo de passar muitos parâmetros indica necessidade de modelagem por objeto.

Comparativo técnico: array vs ArrayList

`array` é melhor quando:

- tamanho é conhecido e estável;
- há foco em simplicidade estrutural e acesso direto por índice.

`ArrayList` é melhor quando:

- tamanho varia durante execução;
- há necessidade de inserir/remover sem redesenhar estrutura base.

Pergunta de decisão de arquitetura (nível júnior/pleno):

- o domínio tem cardinalidade previsível ou dinâmica?
- essa resposta define boa parte da escolha de estrutura.

Perguntas de decisão

1. Qual vantagem da lista de objetos?
2. Quando `get(índice)` pode ser frágil?
3. O que o `for-each` simplifica?
4. Qual problema de escalabilidade o `ArrayList` resolve nesta etapa?
5. O que o `<Conta>` protege no uso da coleção?
6. Em que cenário um `array` fixo ainda faria mais sentido?

Variações de cenário

1. Adicionar terceira conta.
2. Trocar ordem das contas e observar impacto no `get`.

Exercícios progressivos

1. Aplicar depósito em todas as contas com `for-each`.
2. Exibir apenas contas com saldo acima de 800.
3. Reescrever o cadastro inicial com `Conta[]` e listar diferenças de manutenção.
4. Explicar, em 4 linhas, por que `ArrayList<Conta>` melhora o fluxo desta aula.

Desafio de equipe

Criar método para listar somente **id** e **saldo** de cada conta.

Critérios de aceite dos exercícios

- Regra principal continua válida após alterações.
- Cenário de erro foi testado e tratado sem quebrar execução.
- Código permanece legível e coerente com o nível da aula.

Checklist de progressividade técnica

- Pré-requisito confirmado: leitura e execução do código-base da aula anterior.
- Novidade técnica identificada: explicar em uma frase o recurso novo desta aula.
- Complexidade controlada: apontar qual decisão de projeto evita acoplamento desnecessário.
- Evidência prática: executar ao menos um caso de sucesso e um caso de erro no Tryit.

Fechamento executivo

Com coleção de objetos pronta, o próximo passo é encontrar conta específica sem depender de posição fixa.

Revisão rápida (5 minutos)

1. Regra central: qual condição decide aprovação ou bloqueio na aula?
2. O que o **ArrayList** mudou no fluxo operacional em relação ao **array** fixo?
3. Qual risco ainda permanece mesmo com coleção dinâmica?
4. Evolução: qual pequena extensão agregaria mais valor sem aumentar acoplamento?

Mini-missão de fechamento (5 min):

- Execute um cenário de borda no Tryit e justifique o resultado em 3 linhas.

Execução no W3Schools Tryit

1. Abrir o compilador online: https://www.w3schools.com/java/tryjava.asp?filename=demo_compiler
2. Colar todo o conteúdo de AppGestaoContas.java no editor.
3. Clicar em Run, registrar saída base e depois testar variações e exercícios.
4. Manter uma aba separada para cada exercício e comparar resultados.

Referências W3Schools da aula

- https://www.w3schools.com/java/java_arraylist.asp
- https://www.w3schools.com/java/java_for_loop.asp
- https://www.w3schools.com/java/java_compiler.asp

Leituras complementares

- <https://docs.oracle.com/javase/tutorial/>
- <https://dev.java/learn/>